



Université Hassan 1^{er}
Faculté des sciences et
techniques
Settat

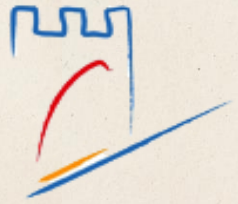


RAPPORT SIMULATEUR MOTOROLA 6809

Java

MAKIL YASSINE
ANAS MARGHAD
ADAM MARS

Encadre par : M. BENALLA
2025/2026 LST GI group2



RAPPORT : Simulateur motorola 6809

Projet de module Architect d'ordinateur

SOMMAIRE

1	Introduction	p : 3
2	DEFINITION	p : 4
3	MOTO6809	P : 5
4	LES FENETRES PRINCIPAUX	P : 7
5	INSTRUCTIONS	P : 11
6	Conclusion	p : 13

☒ INTRODUCTION

Dans le cadre de ce projet, nous avons utilisé un simulateur du microprocesseur Motorola 6809, un processeur 8 bits reconnu pour sa puissance et son avancée technologique dans les années 1970. Ce simulateur logiciel offre la possibilité de reproduire fidèlement le comportement du processeur, permettant ainsi de développer, tester et déboguer des programmes en assembleur sans nécessiter de matériel physique.

L'objectif principal de ce projet est d'explorer les fonctionnalités du microprocesseur Motorola 6809 à travers la programmation et l'exécution de différents algorithmes. En exploitant les capacités du simulateur, nous avons pu analyser les instructions, gérer la mémoire, et observer pas à pas le fonctionnement du code. Ce travail nous a également permis de mieux comprendre les concepts fondamentaux des microprocesseurs et leur rôle dans les systèmes embarqués, tout en renforçant nos compétences en programmation bas-niveau.

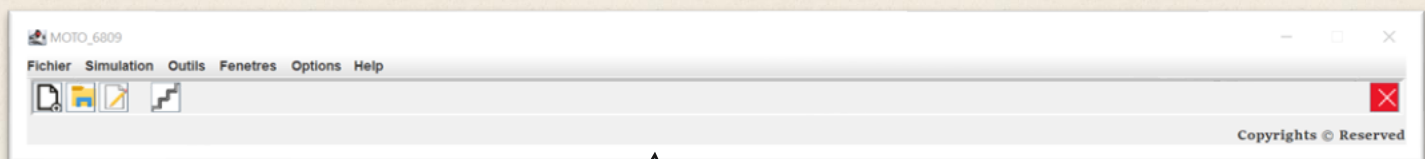
RAPPORT :

Simulateur motorola 6809

Projet de module Architect d'ordinateur

☑ DEFINITION

Le simulateur de moto6809 est un logiciel conçu pour émuler et tester des programmes écrits en langage assembleur spécifiquement pour le processeur Motorola 6809. Il dispose d'une interface graphique qui simplifie la simulation ainsi que le débogage des programmes.



Fenêtre D'ICONES

Fenêtre RAM

Fenêtre ROM

Fenêtre PROGRAMME

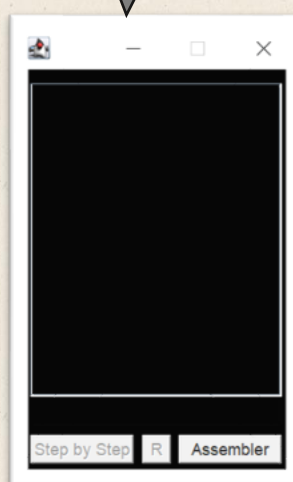
Fenêtre ARCHETERCURE

RAM Memory

dresse 0000	04
dresse 0001	18
dresse 0002	18
dresse 0003	00
dresse 0004	00
dresse 0005	00
dresse 0006	00
dresse 0007	00
dresse 0008	00
dresse 0009	00
dresse 000A	00
dresse 000B	00
dresse 000C	00
dresse 000D	00
dresse 000E	00
dresse 000F	00
dresse 0010	00
dresse 0011	00
dresse 0012	00
dresse 0013	00
dresse 0014	00

ROM

dresse 8000	B6
dresse 8001	00
dresse 8002	00
dresse 8003	BB
dresse 8004	00
dresse 8005	01
dresse 8006	F6
dresse 8007	00
dresse 8008	02
dresse 8009	F0
dresse 800A	00
dresse 800B	00
dresse 800C	B0
dresse 800D	00
dresse 800E	02
dresse 800F	B7
dresse 8010	00
dresse 8011	03
dresse 8012	BE
dresse 8013	00
dresse 8014	03



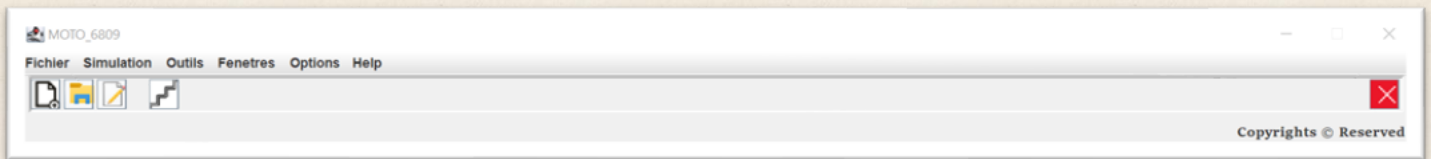
CPU Monitor - 6809

Registres généraux		Pointeurs	
A	00	PC	8000
B	00	S	0000
X	0000	U	0000
Y	0000	DP	00
Flags (E F H I N Z V C)			
00000100			

☒ MOTO6809

La fenêtre principale du logiciel comporte plusieurs éléments distincts :

1.Barre de Menu :



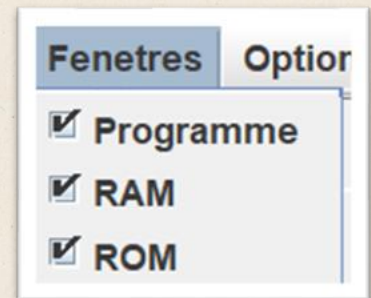
FICHER :

- **Nouveau**  : Cette option permet de créer un nouveau fichier assembleur. Elle est idéale lorsque vous démarrez un nouveau projet ou souhaitez rédiger un programme depuis le début.
- **Ouvrir**  : Permet de charger un fichier source existant dans le simulateur. Cela peut inclure des fichiers assembleur ou binaires à modifier ou exécuter.

- **Enregistrer**  : Sauvegarde les modifications effectuées sur le fichier source actif. Utile pour éviter de perdre votre travail en cours.
- **Quitter**  : Cette option ferme le simulateur une fois que vous avez terminé votre session ou travail. Une confirmation est généralement demandée pour sauvegarder les changements avant de quitter.

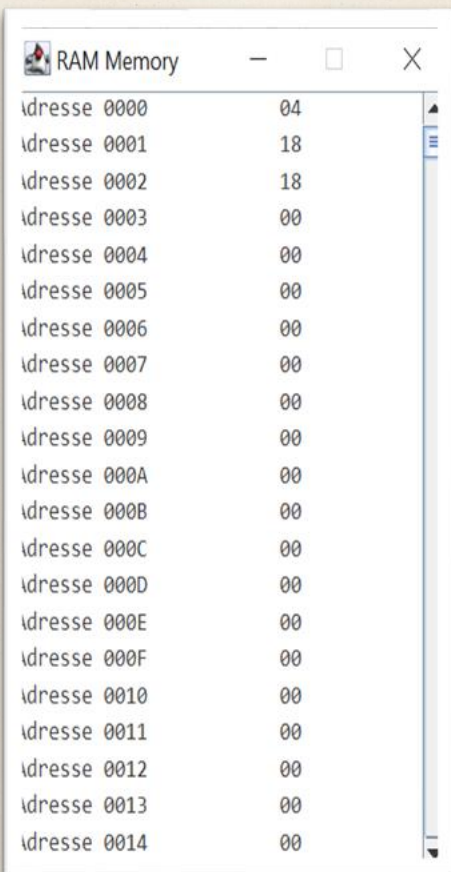
FENETRE :

Le menu Fenêtre du simulateur Moto6809 permet d'accéder à différentes vues essentielles pour analyser et gérer les éléments du programme et de la mémoire (Programme, RAM, ROM).



☑ LES FENETRES PRINCIPALES

➤ *RAM*



Adresse	Valeur (hexadécimale)
0000	04
0001	18
0002	18
0003	00
0004	00
0005	00
0006	00
0007	00
0008	00
0009	00
000A	00
000B	00
000C	00
000D	00
000E	00
000F	00
0010	00
0011	00
0012	00
0013	00
0014	00

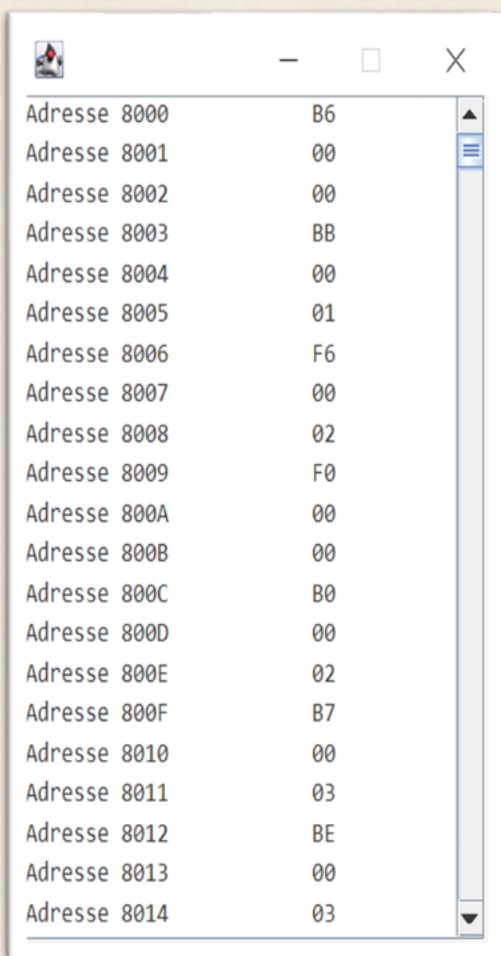
La fenêtre **RAM** (Random Access Memory) affiche la mémoire vive du simulateur. Elle présente chaque adresse de 16 bits avec sa valeur stockée (en hexadécimal) et son équivalent **ASCII**, la RAM est essentielle au microprocesseur car elle sert de zone de travail rapide pour le stockage des variables et des résultats intermédiaires, garantissant des cycles de lecture/écriture indispensables à l'exécution fluide du code. Elle assure également la gestion dynamique de la pile (Stack) pour les appels de sous-programmes et la sauvegarde des registres, offrant ainsi la flexibilité nécessaire à la manipulation de données changeantes, contrairement à la ROM qui reste fixe

RAPPORT :

Simulateur motorola 6809

Projet de module Architect d'ordinateur

➤ *ROM*



Adresse 8000	B6
Adresse 8001	00
Adresse 8002	00
Adresse 8003	BB
Adresse 8004	00
Adresse 8005	01
Adresse 8006	F6
Adresse 8007	00
Adresse 8008	02
Adresse 8009	F0
Adresse 800A	00
Adresse 800B	00
Adresse 800C	B0
Adresse 800D	00
Adresse 800E	02
Adresse 800F	B7
Adresse 8010	00
Adresse 8011	03
Adresse 8012	BE
Adresse 8013	00
Adresse 8014	03

La fenêtre ROM : La fenêtre "ROM" du simulateur affiche le contenu de la mémoire morte, une mémoire non volatile utilisée pour stocker des données permanentes. Elle contient principalement le programme principal, regroupant les instructions nécessaires à l'exécution des tâches principales, ainsi que les instructions système essentielles au bon fonctionnement du simulateur et à son démarrage. Cette fenêtre permet de visualiser et de vérifier ces données fixes dans le cadre de l'émulation

➤ *PROGRAMME*

RAPPORT :

Simulateur motorola 6809

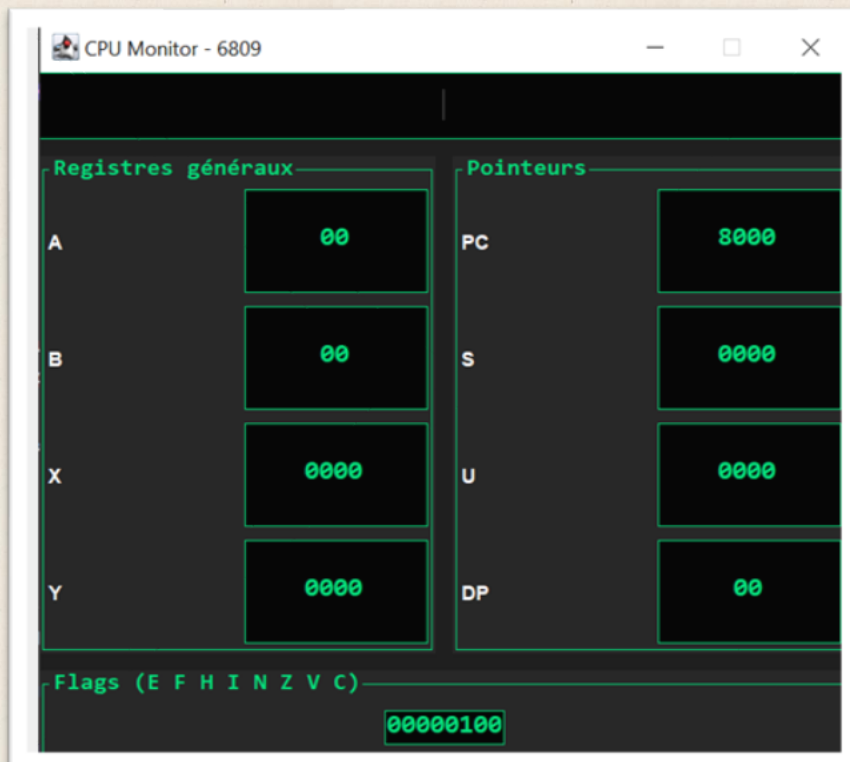
Projet de module Architect d'ordinateur



La fenêtre PROGRAMME : La fenêtre "Programme" permet d'afficher le code machine actuellement exécuté par le processeur, offrant ainsi la possibilité de suivre l'exécution du programme étape par étape ou d'observer son comportement en temps réel. De plus, le menu "Fenêtre" propose une option intitulée "Arranger". Cette fonctionnalité permet d'organiser les trois fenêtres principales ("Programme", "RAM" et "ROM") côte à côte.

- **Step by Step :** Exécute le programme une instruction à la fois pour analyser son comportement.
- **Assembler :** Traduit le code assembleur en code machine exécutable.
- **R :** (RESET) Réinitialise le système.

➤ *ARCHITECTURE*



Cette fenêtre présente non seulement l'architecture interne du 6809, mais aussi le contenu des différents registres internes lors de la simulation.

Elle permet en outre une modification temporaire du contenu des registres A, B, DP, S, U, X et Y.

- **A, B (Accumulateurs) :** Registres de 8 bits fondamentaux pour les opérations arithmétiques. Ils peuvent être fusionnés pour former le registre **D** (16 bits).
- **DP (Pointeur de Page) :** Désigne la page active pour un accès optimal à la mémoire.

S, U (Pointeurs de Piles) : 16 bits. Le registre S est réservé aux instructions système, tandis que le registre U est à disposition de l'utilisateur.

- **X, Y (Registres Généraux) :** Dotés d'une taille de 16 bits, ces registres permettent la manipulation de deux piles distinctes.

Le registre S est réservé aux instructions système, notamment les appels de sous-programmes, tandis que le registre U est à disposition de l'utilisateur pour des tâches personnalisées telles que le passage de paramètres.

☒ **Instructions :**

- **LDA :** Chargement du registre A avec le contenu mémoire
- **LDB :** Chargement du registre B avec le contenu mémoire
- **LDD :** Chargement du registre D avec le contenu mémoire
- **LDS :** Chargement du registre S avec le contenu mémoire
- **LDU :** Chargement du registre U avec le contenu mémoire
- **LDX :** Chargement du registre X avec le contenu mémoire
- **LDY :** Chargement du registre Y avec le contenu mémoire
- **STA :** Stockage du registre A dans la cellule mémoire
- **STB :** Stockage du registre B dans la cellule mémoire
- **STD :** Stockage du registre D dans la cellule mémoire
- **STS :** Stockage du registre S dans la cellule mémoire
- **STU :** Stockage du registre U dans la cellule mémoire

RAPPORT :

Simulateur motorola 6809

Projet de module Architect d'ordinateur



- **STX** : Stockage du registre X dans la cellule mémoire
- **STY** : Stockage du registre Y dans la cellule mémoire
- **ADDA** : Addition du contenu mémoire au registre A
- **ADDB** : Addition du contenu mémoire au registre B
- **ADDD** : Addition du contenu mémoire au registre D
- **ANDA** : ET logique entre le registre A et le contenu mémoire
- **ANDB** : ET logique entre le registre B et le contenu mémoire
- **SUBA** : Soustraction du contenu mémoire du registre A
- **SUBB** : Soustraction du contenu mémoire du registre B
- **SUBD** : Soustraction du contenu mémoire du registre D
- **DECA** : Décrémentation du registre A
- **DECB** : Décrémentation du registre B
- **INCA** : Incrémentation du registre A
- **INCB** : Incrémentation du registre B
- **CLRA** : Remise à zéro du registre A
- **CLRB** : Remise à zéro du registre B
- **CLR** : Remise à zéro d'une case mémoire
- **ABX** : Addition du registre B au registre X
- **ASLA** : Décalage arithmétique à gauche du registre A
- **ASLB** : Décalage arithmétique à gauche du registre B
- **ASL** : Décalage arithmétique à gauche du contenu mémoire
- **LSLA** : Décalage logique à gauche du registre A
- **LSLB** : Décalage logique à gauche du registre B
- **LSL** : Décalage logique à gauche du contenu mémoire
- **LSRA** : Décalage logique à droite du registre A
- **LSRB** : Décalage logique à droite du registre B
- **LSR** : Décalage logique à droite du contenu mémoire
- **ASRA** : Décalage arithmétique à droite du registre A
- **ASRB** : Décalage arithmétique à droite du registre B
- **ASR** : Décalage arithmétique à droite du contenu mémoire
- **ORA** : OU logique entre le registre A et le contenu mémoire

- **ORB** : OU logique entre le registre B et le contenu mémoire
- **PSHS / PSHU** : Empile (Push) les registres sélectionnés sur la pile S ou U
- **PULS / PULU** : Dépile (Pull) les valeurs de la pile vers les registres sélectionnés
- **SWI** : Interruption logicielle
- **NOP** : Pas d'opération (incrémentation du compteur de programme PC)
- **END** : Marque la fin du programme et l'arrêt de l'exécution des instructions.

CONCLUSION

En conclusion, ce guide a offert une exploration complète et détaillée de l'utilisation et des fonctionnalités du simulateur Motorola 6809. En parcourant les différentes fenêtres, telles que la fenêtre principale, l'éditeur, l'architecture, la RAM et la ROM, nous avons mis en lumière les outils visuels et interactifs qui permettent de mieux comprendre le fonctionnement interne de ce processeur emblématique. Les codes d'instructions en assembleur ont été expliqués en détail, soulignant leur rôle crucial dans la programmation et l'exécution des programmes. Ce guide fournit aux utilisateurs une base solide pour exploiter pleinement le simulateur, que ce soit dans un cadre éducatif, expérimental ou pour des applications avancées. Grâce à cette ressource, les utilisateurs peuvent s'immerger dans une expérience interactive, favorisant l'apprentissage, l'exploration et la création de projets innovants.

fin

